МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра Математическая кибернетика и информационные технологии

Отчет по лабораторной работе №8.

Выполнил: студент группы ___БВТ2402__
Скопа Михаил Вячеславович

Москва, 2025

Цель работы: познакомиться с созданием аннотаций. Использовать Stream API по заданному условию.

Разработать приложение, которое считывает данные из исходного источника (например, файл, база данных или сетевой ресурс), применяет к ним различные операции с использованием Stream API и сохраняет результаты в новый источник данных.

1. Создайте аннотацию @DataProcessor, которая будет использоваться для пометки методов обработки данных.

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface DataProcessor {
}
```

Создал аннотацию, изменение которой доступно по ходу работы программы. Аннотация будет применяться к методу.

2. Создайте класс DataManager, который будет отвечать за многопоточную обработку данных. Этот класс должен иметь методы:

• registerDataProcessor(Object processor) — регистрирует объект — обработчик данных с аннотацией @DataProcessor;

• loadData(String source) — загружает данные из исходного источника;

```
class DataManager {

    private List<String> data = new ArrayList<>();
    private List<String> processedData = new ArrayList<>();
    private final List<Object> processors = new ArrayList<>();

    private final ExecutorService executor = Executors.newFixedThreadPool(nThreads: 6);

    public void registerDataProcessor(Object processor) {
        processors.add(processor);
    }

    public void loadData(String source) throws IOException {
        data = Files.readAllLines(Path.of(source));
    }
```

Создал класс DataManager с методами registerDataProcessor и loadData. Для работы создал фиксированное число потоков: 6.

• processData() — запускает многопоточную обработку данных, применяя методы с аннотацией @DataProcessor с использованием Stream API;

```java
public void processData() throws Exception {

    List<Callable<List<String>>> tasks = new ArrayList<>();

    for (Object processor : processors) {
        for (Method method : processor.getClass().getDeclaredMethods()) {
            if (method.isAnnotationPresent(annotationClass: DataProcessor.class)) {

                method.setAccessible(flag: true);

            tasks.add(() -> {
                @SuppressWarnings("unchecked")
                List<String> result =
                        (List<String>) method.invoke(processor, data);

                List<String> withMarker = new ArrayList<>();

                withMarker.add(
                    "=== " + processor.getClass().getSimpleName()
                    + "." + method.getName() + " ==="
                );

                withMarker.addAll(result);
                withMarker.add(e: "");

                return withMarker;
            });
            }
        }
    }

    List<Future<List<String>>> results = executor.invokeAll(tasks);

    processedData = results.stream()
            .flatMap(f -> {
                try {
                    return f.get().stream();
                } catch (Exception e) {
                    throw new RuntimeException(e);
                }
            })
            .collect(Collectors.toList());
}
```

Создал метод processData. Метод сначала создает список задач, далее для каждого заданного процесса перебираю все методы, для методов с созданной аннотацией создаю задачу с этим методом. Также добавляю разделители. Потом создаю список с результатами от каждого таска, считываю и удобно записываю с помощью Stream API. В итоге возвращается список с результатами выполненных задач.

• saveData(String destination) — сохраняет обработанные данные в новый источник.

```java
public void saveData(String destination) throws IOException {
    Files.write(Path.of(destination), processedData);
    executor.shutdown();
}
```

Метод для сохранения данных в файл и прекращение работы с потоками.

2. Создайте несколько классов, представляющих различные обработчики данных, и пометьте их аннотацией @DataProcessor. Например, можно создать классы для фильтрации, трансформации и агрегации данных.

```java
class FilterProcessor {

    @DataProcessor
    public List<String> filter(List<String> data) {
        return data.stream()
                .filter(s -> s.length() > 3)
                .collect(Collectors.toList());
    }
}

class TransformProcessor {

    @DataProcessor
    public List<String> transform(List<String> data) {
        return data.stream()
                .map(String::toUpperCase)
                .collect(Collectors.toList());
    }
}

class AggregateProcessor {

    @DataProcessor
    public List<String> aggregate(List<String> data) {
        return List.of("TOTAL ELEMENTS: " + data.size());
    }
}
```

Создал методы с аннотациями для фильтрации, трансформации и агрегации.

3. Используйте многопоточность из java.util.concurrent для эффективной обработки данных параллельно.
4. Протестируйте ваше приложение, загрузив данные из исходного источника, применив различные обработчики с помощью Stream

API, и сохраните результаты в новый источник

```java
public class Main {

    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws Exception {

        DataManager manager = new DataManager();

        manager.registerDataProcessor(new FilterProcessor());
        manager.registerDataProcessor(new TransformProcessor());
        manager.registerDataProcessor(new AggregateProcessor());

        manager.loadData(source: "input.txt");
        manager.processData();
        manager.saveData(destination: "output.txt");

        System.out.println(x: "Обработка завершена. Результат сохранён в output.txt");
    }
}
```

Создал демонстрационный метод. Создал экземпляр класса DataManager, задал три процесса: фильтрация, трансформация и агрегация. Считал входной файл, запустил программу, записал ответ в выходной файл.

```
Lab8 > input.txt
api
Loader
Print
Print
```

```
Lab8 > output.txt
=== FilterProcessor.filter ===
Loader
Print
Print

=== TransformProcessor.transform ===
API
LOADER
PRINT
PRINT

=== AggregateProcessor.aggregate ===
TOTAL ELEMENTS: 4
```

Вывод: познакомился с многопоточностью в языке программирования Java. Написал программу, использующую несколько потоков одновременно.

https://github.com/Mirphon/ITIP/tree/main/Lab8